

Introduction to Machine Learning

Lecture 20: Generative Modeling

Ali Bereyhi

`ali.bereyhi@utoronto.ca`

Department of Electrical and Computer Engineering
University of Toronto

Winter 2026

Recap: Generate Coherent Text

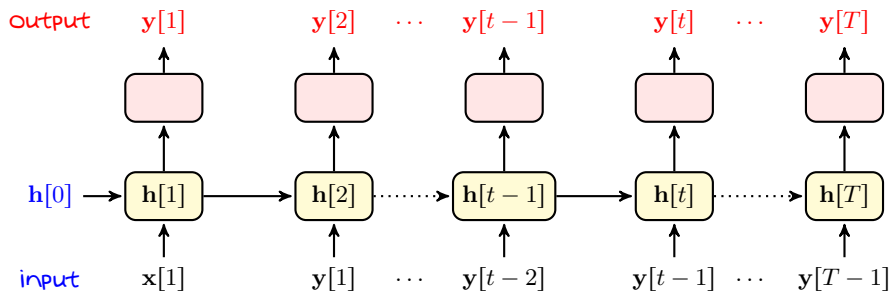
? *Can we make a model generate coherent sentences?*

! *We can use the idea of sequence learning!*

We can construct a supervised learning task

- *Dataset contains many coherent sentences*
 - ↳ *Each sentence is a sequence with each word being $x[t]$*
 - ↳ *Label of each word is the next word*
- *Model is a sequence-based model, e.g., RNN*
- *We minimize the risk computed by cross-entropy*

Recap: Generate Coherent Text



Recap: Alternative Look

In practice, we try to **learn** this distribution using a model

Language Model

Language models learn $P(x[t+1]|x[1:t])$ using a learning model

$$f_{\theta}(x[t+1]|x[1:t])$$

where f_{θ} is realized via a hypothesis, e.g., is set to an RNN

Example: The output of RNN computes a Softmax

- It computes the probability of next word
- If model gets trained properly, it can compute right probability
- We are sampling from this distribution when we infer the model

Today's Agenda: *Generative Modeling*

We continue our final journey which explores briefly

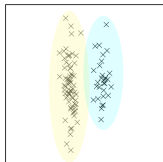
Advanced Topics in Machine Learning

In today's episode, we talk about

- *Data Generative Modeling*

Recap: Extended K -Means Clustering

- In clustering, we could also assume Σ to be **unknown**



- In general, we can assume a distribution $\mathcal{Q}$ that is **mixture of Gaussians**

$$\mathcal{Q} \equiv \sum_{m=1}^M \pi_m \mathcal{N}(\boldsymbol{\mu}_m, \boldsymbol{\Sigma}_m) \text{ with } \sum_{m=1}^M \pi_m = 1$$

- ↳ **Gaussian Mixture Model (GMM)**
- ↳ We estimate $\pi_m, \boldsymbol{\mu}_m, \boldsymbol{\Sigma}_m$ for $m = 1, \dots, M$ all together
- ↳ Larger M makes it better, but needs more computation!

Data Generation

Say x collects all pixel values of an image

$$\mathbb{D} = \{x_n = \text{cat image} : n = 1, \dots, N\}$$



We learn the distribution by finding a *good* estimate $P_\theta(x)$

$$x_{\text{new}} \sim P_\theta(x) \approx P_{\text{data}}(x)$$

x_{new} is a new cat image!



In practice, we need more sophisticated method to estimate $P_{\text{data}}(x)$

More Abstract Viewpoint

? *Can we generate something else, as well?*

! *Sure! We only need to sample correctly*

Say we want to generate a cat image of size $3 \times 224 \times 224$: we would say that this is a random object $\mathbf{X}$ from distribution $P_{data}(\mathbf{X})$

Data Distribution

$P_{data}(\mathbf{X})$ is the distribution of all images $\mathbf{X}$ that are cat, i.e.,

$$P_{data}(\mathbf{X} = \text{DOG}) = 0$$

If we know $P_{data}(\mathbf{X})$ and are able to sample from it, we could

generate a sample cat image by drawing a sample from $P_{data}(\mathbf{X})$

More Abstract Viewpoint

? *Can we find $P_{data}(\mathbf{X})$?*

! *No! But, we can learn it from its samples!*

Any cat image $\mathbf{X}_n$ is a sample from $P_{data}(\mathbf{X})$: we put them in dataset

$$\mathbb{D} = \{\mathbf{X}_1, \dots, \mathbf{X}_N\}$$

We then try learning $P_{data}(\mathbf{X})$ using a model $f_{\theta}(\mathbf{X})$

- *We compute the likelihood of dataset*
- *We find θ that $f_{\theta}(\mathbf{X})$ is close to $P_{data}(\mathbf{X})$*

Approach I: Learning Distribution Directly

? But we don't know $P_{data}(\mathbf{X})$?!

! We can do it without knowing $P_{data}(\mathbf{X})$ explicitly

Using $\mathbb{D}$, we can compute the likelihood of θ

$$\mathcal{L}_{\mathbb{D}}(\theta) = \prod_n f_{\theta}(\mathbf{X}_n)$$

Maximum Likelihood Estimation

Find θ such that the likelihood is maximized

$$\theta^{\star} = \underset{\theta}{\operatorname{argmax}} \log \mathcal{L}_{\mathbb{D}}(\theta)$$

Approach I: Learning Distribution Directly

If we have **enough** samples and **complex** model; then, we see that

$$f_{\theta}(\mathbf{X}) \approx P_{data}(\mathbf{X})$$

However, we should note that

- **Enough** is too much and **complex** is too complicated
- Even if we get a good approximation, we still need to sample it
 - ↳ We may use Monte-Carlo or Gibbs sampling
 - ↳ They can yet be slow or imprecise

This approach was one of first approaches for learning $P_{data}(\mathbf{X})$ that is used in

↳ Flow-based models, energy-based models, . . .

Approach II: Sampling From Data Distribution

People gradually started to think

? Why do not we learn to *directly sample* from $P_{data}(\mathbf{X})$?!

Say we have a noise process like $\mathbf{Z}$: if we pass this random process it through a function $f(\mathbf{Z})$; then, we get an output $\mathbf{X} = f(\mathbf{Z})$

for known noise distribution, the distribution of $\mathbf{X}$ depends on $f(\cdot)$

If we design $f(\cdot)$ properly, we can make $\mathbf{X}$ to be distributed with $P_{data}(\mathbf{X})$!

Approach II: Sampling From Data Distribution

Learn to Sample

Starting from a noise process $\mathbf{Z}$, we approximate a function $f(\cdot)$ by a model $f_{\theta}(\cdot)$, such that $\mathbf{X} = f_{\theta}(\mathbf{Z})$ is approximately distributed with $P_{data}(\mathbf{X})$

This approach simplifies the process significantly: to generate a sample $\mathbf{X}$

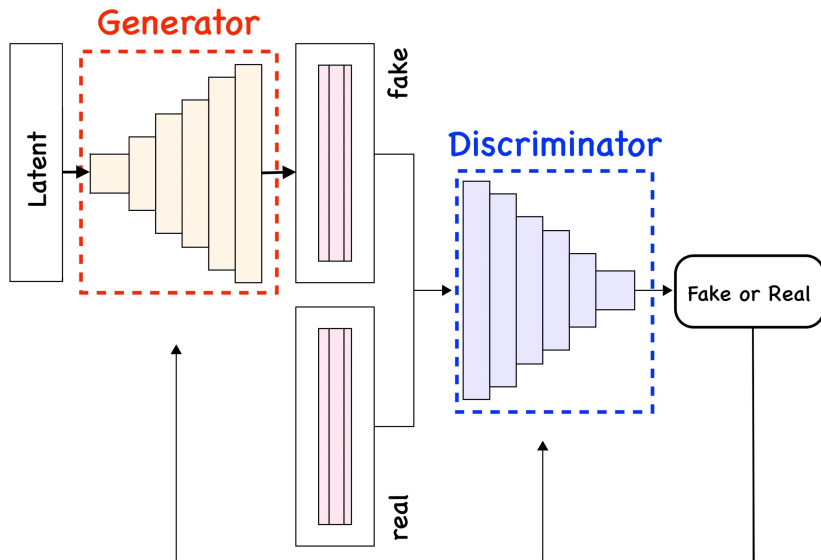
- We sample noise $\mathbf{Z}$*
 - ↳ This is easy! For instance, we sample a Gaussian distribution!*
- We pass the noise sample through the trained model as $\mathbf{X} = f_{\theta}(\mathbf{Z})$ and get the data sample*

Approach II: *Sampling From Data Distribution*

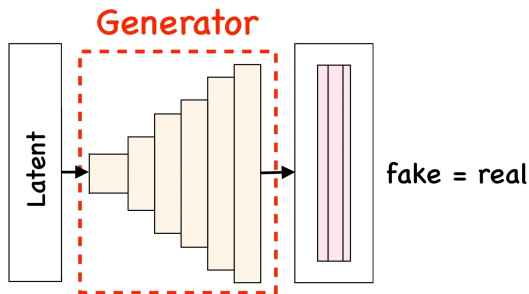
There are three well-known approaches to learn sampling

- *Generative Adversarial Networks (GANs)*
 - ↳ *They learn to sample by a min-max game*
- *Variational Auto-Encoders (VAEs)*
 - ↳ *They use auto-encoding to learn sampling*
- *Diffusion Models*
 - ↳ *They learn sampling through diffusion process*

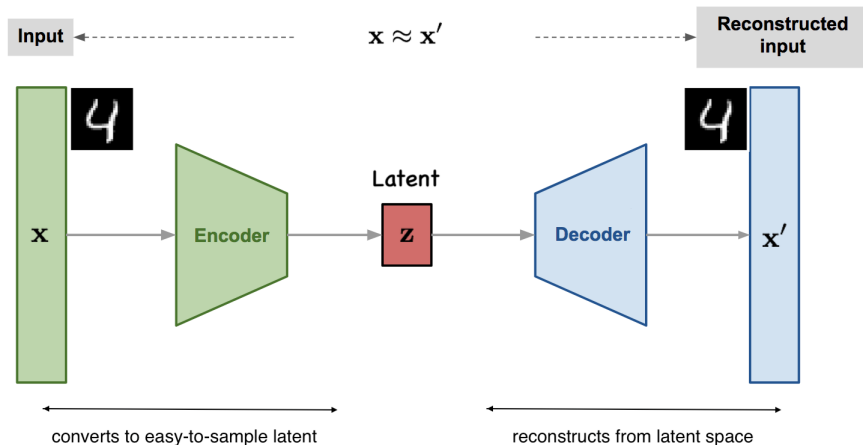
Generative Adversarial Networks (GANs): *Training*



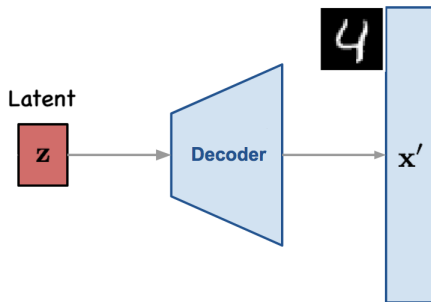
Generative Adversarial Networks (GANs): Generation



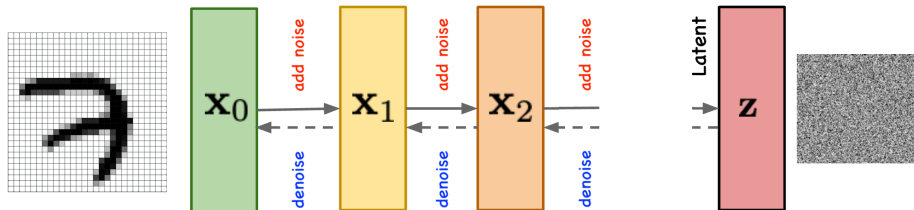
Variational Auto-Encoders (VAEs): Training



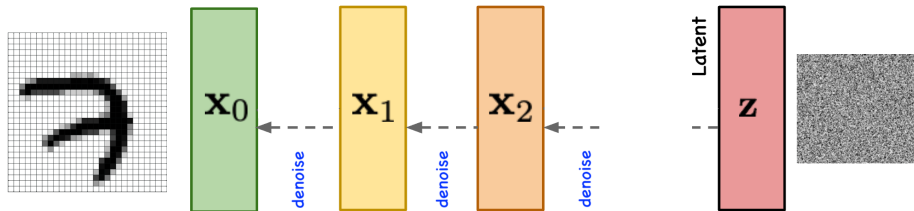
Variational Auto-Encoders (VAEs): *Generation*



Diffusion Models: *Training*



Diffusion Models: *Generation*



Generative vs Discriminative Models

What we used throughout the course for classification were *discriminative models* which compute $P(v|\mathbf{x})$. We could use *generative models* as well to classify: we learn $P(\mathbf{x}, v)$ from samples in dataset and then write

$$P(v|\mathbf{x}) = \frac{P(\mathbf{x}, v)}{P(\mathbf{x})}$$

We however look for v with highest $P(v|\mathbf{x})$, so we have

$$\operatorname{argmax}_v P(v|\mathbf{x}) = \operatorname{argmax}_v P(\mathbf{x}, v)$$

So, we don't need to compute $P(\mathbf{x})$ anymore and can directly use $P(\mathbf{x}, v)$!

- The simplest classifier of this form is the *naive Bayes classifier*
- For basic classification tasks, we still prefer *discriminative models*

Further Read

- Goodfellow
 - ↳ Chapter 20

Generative Models